GDD TAC III - PyShooter

Desenvolvedores:

Guilherme Silva Sampaio: Correções de arquitetura, criação e revisão do GDD, melhorias no código, geração UML, refatoração e testes.
Pedro Henrique Gonçalves Mota: Criação do projeto base, obtenção dos assets para o jogo, e adições de funcionalidades novas ao projeto base, criação e revisão GDD.

1. Game Design Document (GDD)

1.1 Conceito Principal

PyShooter é um jogo de ação 2D side-scrolling onde o jogador assume o papel de um soldado em uma missão de combate através de terrenos naturais hostis: florestas de pinheiros e montanhas. O jogador avança horizontalmente por fases, enfrentando soldados inimigos com armas de fogo e granadas, coletando suprimentos (munição, granadas e kits médicos) e superando obstáculos ambientais como plataformas elevadas e corpos d'água. O objetivo é sobreviver e chegar à saída de cada fase para progredir pelas fases do jogo.

1.2 Estilo / Gênero

Aspecto	**Classificação**
Gênero primário	Run and Gun / Platformer Shooter 2D
Subgênero	Side-scrolling Action
Perspectiva	Vista lateral (side-view)
Referências clássicas	Metal Slug, Contra, Ghosts 'n Goblins

O jogo se encaixa no gênero Run and Gun 2D: o jogador se move horizontalmente, pula entre plataformas e dispara contra inimigos enquanto avança. Não há elementos de puzzle significativos, reforçando o foco na ação direta.

1.3 Plataforma Alvo

Requisito	**Especificação**
Sistema Operacional	Windows (compatível com qualquer SO que suporte Python + Pygame)
Python	3.6+
Dependência	pygame
Resolução de tela	1200 × 600 pixels (fixa)
Periféricos	Teclado (obrigatório), Mouse (apenas nos menus)
Hardware mínimo	Qualquer PC capaz de rodar Python; sem exigências de GPU dedicada

1.4 Mecânicas e Física

1.4.1 Mapa Completo de Controles

Controles de Jogo (In-Game)

Tecla	**Ação**
A ou ← (seta esquerda)	Mover para a esquerda
D ou → (seta direita)	Mover para a direita
W ou ↑ (seta cima)	Pular
Espaço	Atirar (manter pressionado para tiro contínuo)
Q ou G	Lançar granada
ESC	Fechar o jogo

Controles de Menu

Periférico	**Ação**
Mouse (clique esquerdo)	Clicar nos botões Start, Controls, Resume, Options, Exit, Restart e Seleção da Fase

Controles do Level Editor (level-editor.py)

Tecla	**Ação**
← / →	Scroll do mapa no editor
↑ / ↓	Mudar número do nível
Shift Direito	Aumentar velocidade do scroll (5x)
Mouse esquerdo	Pintar tile selecionado
Mouse direito	Apagar tile

1.4.3 Habilidades do Personagem Principal

Habilidade	**Descrição**	**Parâmetros**
Movimento	Caminhar em ambas as direções	Velocidade: 5 px/frame
Pulo	Pulo simples (sem pulo duplo)	Impulso: -11; gravidade: 0.75
Tiro	Disparar projéteis horizontais	Cooldown: 20 frames; dano: 25 HP (inimigos), 5 HP (jogador); munição inicial: 20
Granada	Arremesso em arco parabólico	Dano: 50 HP em área; raio: 2 tiles; timer: 100 frames; estoque inicial: 5
Coleta de itens	Pegar caixas de suprimentos	Vida: +25 HP, Munição: +15, Granadas: +3

1.5 Assets (Arte e Som)

1.5.1 Sprites Mapeados

Personagens (Spritesheet por frames individuais)

Entidade	**Animações**	**Frames por animação**	**Escala**
Player (img/player/)	Idle, Run, Jump, Death	5, 6, 1, 8	1.65x
Enemy (img/enemy/)	Idle, Run, Jump, Death	Mesma estrutura	1.65x
SkeletonEnemy (`img/skeleton/`)	Idle, Run, Shoot, Death	7, 8, 14, 5	1.65x
RobotEnemy (img/robot/)	Idle, Walk, Attack, Death, Hurt	4, 4, 4, 4, 2	1.55x
SniperEnemy (img/sniper/)	Idle, Walk, Shot_1, Dead, Hurt	7, 8, 4, 5, 4	1.2x
DemonBoss (img/enemy/Demon/)	Idle, Walk, Attack, Death	3, 6, 4, 6	1.5x
DragonBoss (img/enemy/Dragon/)	Idle, Walk, Attack, Hurt, Death	3, 5, 4, 2, 5	1.5x

Cenário e Tiles

ID do Tile	**Tipo**	**Função no jogo**
0–8, 24–35, 47–48, 50, 58–61, 65–66, 73–77, 83–84	Obstáculos sólidos	Plataformas e paredes com colisão ativa (Variações de Terra, Neve, Pedras e Grama).
9–10, 37–39, 42–46, 51–57, 62–64, 67–68	Água	Risco ambiental (Mata o jogador/inimigo ao contato).
11–14, 36, 40–41, 49, 71–72, 78–82	Decoração	Elementos visuais sem colisão (Mato, Boneco de

ID do Tile	Tipo	Função no jogo
		Neve, Árvores Gigantes, Placas, Pinheiros).
15	Spawn do jogador	Posição inicial do player
16	Spawn do inimigo (Soldado)	Define a posição inicial do inimigo patrulheiro básico.
17	Caixa de munição	Item coletável (+15 ammo)
18	Caixa de granada	Item coletável (+3 granadas)
19	Caixa de vida	Item coletável (+25 HP)
20	Saída	Portal de fim de fase
69	Spawn de Inimigo (Robô)	Posição inicial do RobotEnemy (Atirador vetorial).
70	Spawn de Inimigo (Esqueleto)	Posição inicial do SkeletonEnemy (Arqueiro).
85	Rampa	Bloco de terreno com colisão diagonal/inclinada
86	Spawn do DemonBoss	Posição inicial do chefe terrestre de curto alcance.
87	Spawn do DragonBoss	Posição inicial do chefe voador.

Background (Parallax)

Arquivo	Descrição
sky_cloud.png	Céu com nuvens (camada mais distante)
back.png	Céu claro com silhueta de cordilheiras azuis (camada distante alternativa)
mountain.png	Silhueta de montanhas
pine1.png	Pinheiros distantes

Arquivo	Descrição
pine2.png	Pinheiros próximos (camada frontal)
background4.png	Floresta densa e pântano (Background exclusivo da Fase 4 / Arena dos Bosses)

Ícones e Projéteis

Arquivo	Uso
bullet.png	Sprite do projétil e indicador de munição no HUD
grenade.png	Sprite da granada e indicador no HUD
health_box.png / ammo_box.png / grenade_box.png	Caixas de itens coletáveis
Ball1.png / Ball2.png	Sprites do projétil de energia disparado pelo inimigo Robô (img/robot/)

Explosão

Arquivo	Descrição
exp1.png – exp5.png	5 frames de animação de explosão (escala 0.5x)
Fire_Attack1.png – Fire_Attack6.png	6 frames de animação do projétil de fogo guiado do DragonBoss (img/enemy/dragon/)

UI (Botões)

Arquivo	Uso
start_btn.png	Botão "Iniciar" no menu principal
exit_btn.png	Botão "Sair" no menu principal
restart_btn.png	Botão "Reiniciar" na tela de morte
save_btn.png / load_btn.png	Botões do Level Editor
pause_btn.png	Botão "Pausar" no canto superior direito do HUD durante a gameplay

1.5.3 Efeitos Sonoros e Música

Arquivo	**Tipo**	**Volume**	**Uso**
music2.mp3	Trilha musical	0.2 (20%)	Loop infinito com fade-in de 5s
shot.wav	SFX	0.05 (5%)	Disparo de projétil
jump.wav	SFX	0.05 (5%)	Pulo do jogador
grenade.wav	SFX	0.05 (5%)	Explosão de granada
robot_shoot.wav	SFX	0.05 (5%)	Disparo de projétil de energia (RobotEnemy)

1.6 - Fórmulas e Mecânicas Matemáticas do PyShooter

1.6.1 Física de Movimentação e Gravidade

Aplicada às entidades que sofrem a ação da gravidade (Jogador, Inimigos e Granadas). O motor do jogo calcula a queda baseando-se no incremento constante da aceleração sobre a velocidade vertical.

- **Fórmula da Gravidade (por frame):**
  `Velocidade_Y = Velocidade_Y + Gravidade`

- **Fórmula do Pulo:**
  `Velocidade_Y = -Força_do_Pulo`

1.6.2. Visão da Inteligência Artificial (Cálculo de Distância)

Para que os inimigos tomem decisões baseadas na posição do jogador (como atirar ou seguir), o jogo mede a distância absoluta em pixels entre os dois centros de massa utilizando o **Teorema de Pitágoras** (através da função `math.hypot`).

- **Fórmula de Distância Euclidiana:**

$$\text{Distância} = \sqrt{(X_{\text{jogador}} - X_{\text{inimigo}})^2 + (Y_{\text{jogador}} - Y_{\text{inimigo}})^2}$$

1.6.3. Mira e Tiros Teleguiados (Trigonometria)

Diferente dos tiros de armamentos padrão (que viajam horizontalmente), projéteis que miram diretamente no jogador (como as flechas atiradas pelos Esqueletos) utilizam trigonometria para calcular a trajetória de interceptação.

- **Cálculo do Ângulo de Tiro:**

$$\theta = \text{atan2}(Y_{\text{jogador}} - Y_{\text{origem}}, X_{\text{jogador}} - X_{\text{origem}})$$

- **Decomposição da Velocidade nos Eixos:**

$$dx = \cos(\theta) \times Velocidade$$

$$dy = \sin(\theta) \times Velocidade$$

1.6.4. Proporção da Interface de Usuário (Barra de Vida)

O preenchimento verde da barra de vida do HUD (Heads-Up Display) varia dinamicamente conforme o jogador sofre dano ou coleta itens médicos. O cálculo é feito utilizando uma razão geométrica.

- **Razão de Vida:**

$$\text{Razão} = \frac{\text{Vida Atual}}{\text{Vida Máxima}}$$

- **Fórmula de Renderização:**

$$\text{Largura Exibida} = \text{Razão} \times \text{Largura Total da Barra}$$

1.6.5. Física Parabólica e Reflexão (Granada)

A granada não descreve uma reta, mas sim um arco parabólico durante seu tempo de fusão, combinando movimento linear e aceleração da gravidade, e é o único projétil no jogo com física de rebatimento horizontal.

- **Movimento do Arco Parabólico:**
  A granada viaja simultaneamente através de um deslocamento fixo no eixo horizontal e um deslocamento acelerado no eixo vertical.
  `dx = Direção * Velocidade`
  `dy = Velocidade_Y` (acumulando a força da gravidade a cada tick)

- **Reflexão Horizontal (Quique em Paredes):**
  Quando o deslocamento previsto em X colide com o *bounding box* lateral de um obstáculo (parede), a granada reflete conservando sua força horizontal no sentido oposto.
  `Direção = Direção * -1`

2. Engenharia de Requisitos

2.1 Requisitos Funcionais (RF)

ID	Requisito
RF-01	O sistema deve permitir que o jogador se mova horizontalmente (esquerda/direita) utilizando as teclas A/D ou setas direcionais.
RF-02	O sistema deve permitir que o jogador pule ao pressionar W ou seta para cima, aplicando gravidade simulada para retorná-lo ao solo.
RF-03	O sistema deve permitir que o jogador dispare projéteis horizontais ao pressionar a barra de espaço, com cooldown de 20 frames e consumo de munição.
RF-04	O sistema deve permitir que o jogador lance granadas (tecla Q/G) que seguem trajetória parabólica, explodem após um timer e causam dano em área.
RF-05	O sistema deve detectar colisão entre projéteis e personagens, aplicando dano diferenciado (25 HP em inimigos, 5 HP no jogador).

ID	Requisito
RF-06	O sistema deve carregar fases a partir de arquivos CSV (levelN_data.csv), interpretando uma matriz 16×150 de IDs de tiles para montar o cenário.
RF-07	O sistema deve implementar IA para inimigos com comportamento de patrulha, detecção de jogador por retângulo de visão e resposta de ataque (tiro).
RF-08	O sistema deve permitir coleta de itens (vida +25, munição +15, granadas +3) ao jogador colidir com caixas de suprimentos.
RF-09	O sistema deve exibir HUD com barra de vida, contagem visual de munição e contagem visual de granadas.
RF-10	O sistema deve transicionar entre 3 fases ao jogador alcançar a saída, com efeito de fade e reset completo dos grupos de sprites.

2.2 Requisitos Não Funcionais (RNF)

ID	Requisito
RNF-01	**Desempenho:** O jogo deve manter taxa de quadros estável de 60 FPS.
RNF-02	**Portabilidade:** O jogo deve rodar em qualquer sistema operacional com Python 3.6+ e Pygame instalados, sem dependências adicionais.
RNF-03	**Usabilidade:** O tempo de carregamento de uma fase deve ser imperceptível (< 1 segundo), com todos os assets carregados em memória na inicialização.
RNF-04	**Manutenibilidade:** Os níveis devem ser editáveis por ferramenta externa (Level

ID	Requisito
	Editor) sem necessidade de alterar código-fonte.
RNF-05	**Consumo de Memória:** O jogo deve funcionar com uso moderado de RAM, utilizando spritesheets individuais e limpeza de sprites via kill() e Group.empty().

3. Arquitetura do Sistema e UML

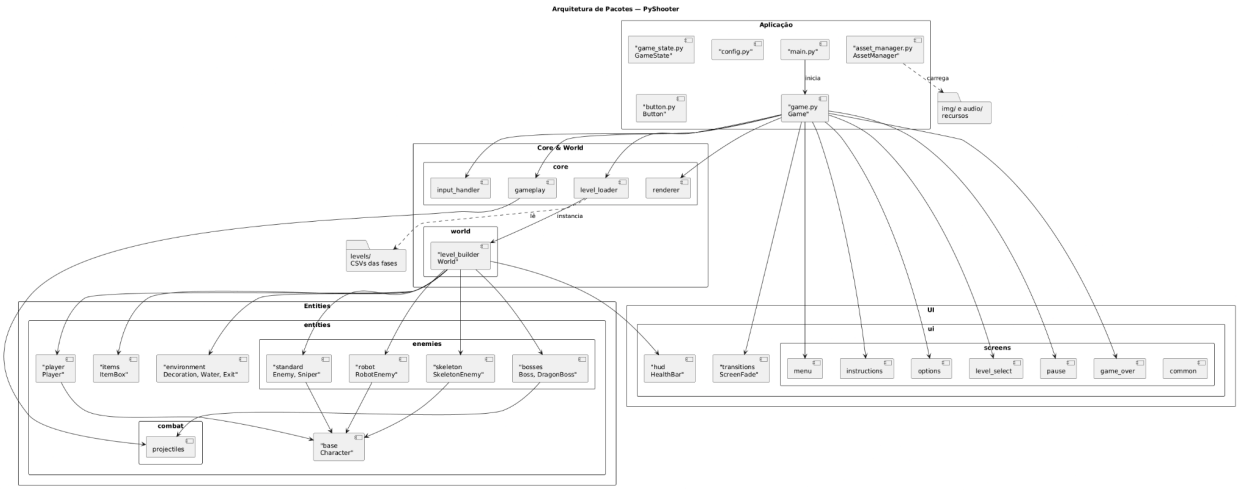
3.1 Diagrama de Módulos (Pacotes)

Descrição da Estrutura:
O projeto adota uma arquitetura modularizada, separando as responsabilidades de ciclo de vida, lógica de mundo, entidades de jogo e interface gráfica em pacotes específicos:

- **Aplicação:** Contém o ponto de entrada (main.py), a classe orquestradora do ciclo de jogo (game.py / Game), a máquina de estados (game_state.py / GameState), as configurações globais (config.py), o gerenciador centralizado de recursos (asset_manager.py / AssetManager) e utilitários de interface (button.py / Button).
- **Core & World:**
  - core/: Contém os subcomponentes de captura de entrada (input_handler), regras de jogo (gameplay), carregamento de níveis (level_loader) e renderização (renderer).
  - world/: Contém o construtor do mapa (level_builder / World), responsável por processar as matrizes dos arquivos CSV das fases.
- **Entities:** Organizado em submódulos dentro do pacote entities/:
  - base: Classe base comum Character.
  - player: Implementação da classe Player.
  - enemies: Módulos específicos para diferentes inimigos (standard / Enemy e Sniper, robot / RobotEnemy, skeleton / SkeletonEnemy, bosses / Boss e DragonBoss).
  - combat: Gerenciamento dos projéteis de combate (projectiles).
  - items & environment: Gerenciamento de itens coletáveis (ItemBox) e elementos ambientais (Decoration, Water, Exit).
- **UI:** Contém os módulos da interface gráfica (ui/):
  - hud: Exibição de barras de estado (HealthBar).
  - transitions: Efeitos visuais de transição de tela (ScreenFade).
  - screens: Telas completas de navegação (menu, instructions, options, level_select,

pause, game_over, common).

- **Recursos externos:** Mapeados pelos diretórios levels/ (matrizes CSV), img/ e audio/.



3.2 Diagrama de Dependências (Classes UML)

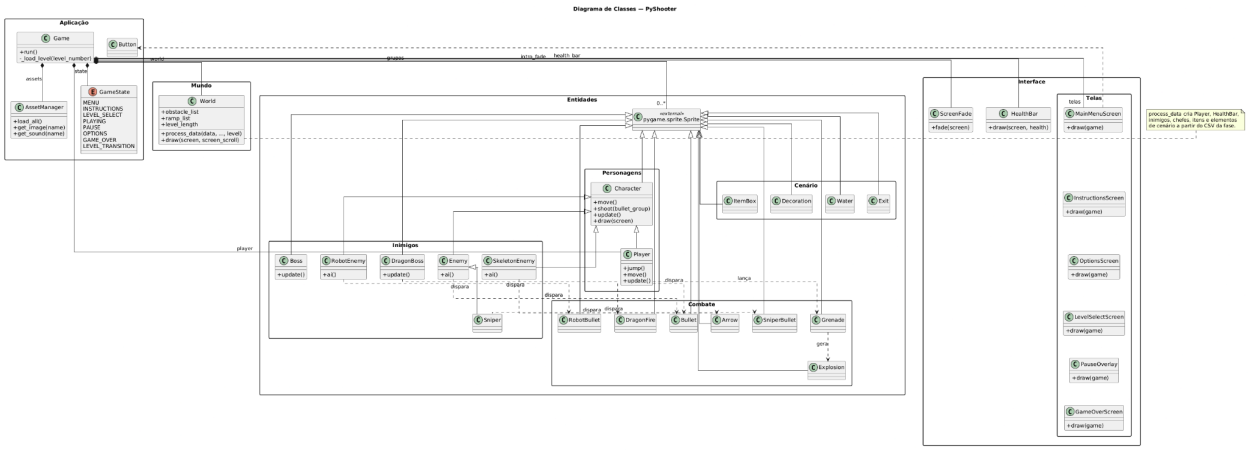
Análise da Orientação a Objetos:

O sistema utiliza um modelo Orientado a Objetos modularizado. A classe principal Game gerencia o GameState, o AssetManager, a instância do World, o Player e os grupos de sprites do Pygame. Todas as entidades do jogo (personagens, projéteis, itens e elementos do cenário) derivam de pygame.sprite.Sprite. A classe Character serve como classe base para Player e Enemy.

Relacionamentos-Chave:

Relacionamento	Tipo	Descrição
Game → GameState	Composição	Mantém o estado atual da aplicação para alternar fluxos de tela e execução.
Game → AssetManager	Composição	Gerencia o carregamento e fornecimento centralizado de imagens e sons.
Game → World	Composição	Mantém a referência do nível

Relacionamento	Tipo	Descrição
		atual do jogo.
Character → pygame.sprite.Sprite	Herança	Base comum para todas as entidades vivas do jogo.
Player / Enemy → Character	Herança	Especializações de personagens controláveis e Inimigos Padrão/Sniper/Robô/Esqueleto.
World → Entidades do jogo	Criação (Factory)	O método process_data() lê a matriz CSV e instancia Player, Inimigos, Chefes, Itens e Cenário.
Player / Inimigos → Projéteis	Associação Dinâmica	Entidades instanciam seus respectivos projéteis (Bullet, SniperBullet, RobotBullet, Arrow, DragonFire, Grenade).
Grenade → Explosion	Composição Dinâmica	Ao expirar o tempo limite do temporizador, a granada gera uma explosão em área.
Screens → Button	Uso	As telas de interface (MainMenuScreen, OptionsScreen, etc.) utilizam botões interativos de UI.



Link do github com o código-fonte

https://github.com/pedromota05/TrabalhoTACIII-PyShooter.git